

Custom Spartan-6 FPGA DSP Slice

Prepared by Abdulrahman Mahmoud Rady

Contents

Overview	2
Phase 1: Pipeline register	3
RTL	3
Testbench	3
Do file.....	4
Test approach	5
Output.....	6
Phase 2: Parent DSP	7
Testbench	9
DO file	11
Output.....	12
QuestaSim.....	12
QuestaLint.....	13
Vivado	14
Constraints file (brief).....	14
Utilization Report	15
Timing Summary	16
Messages	17

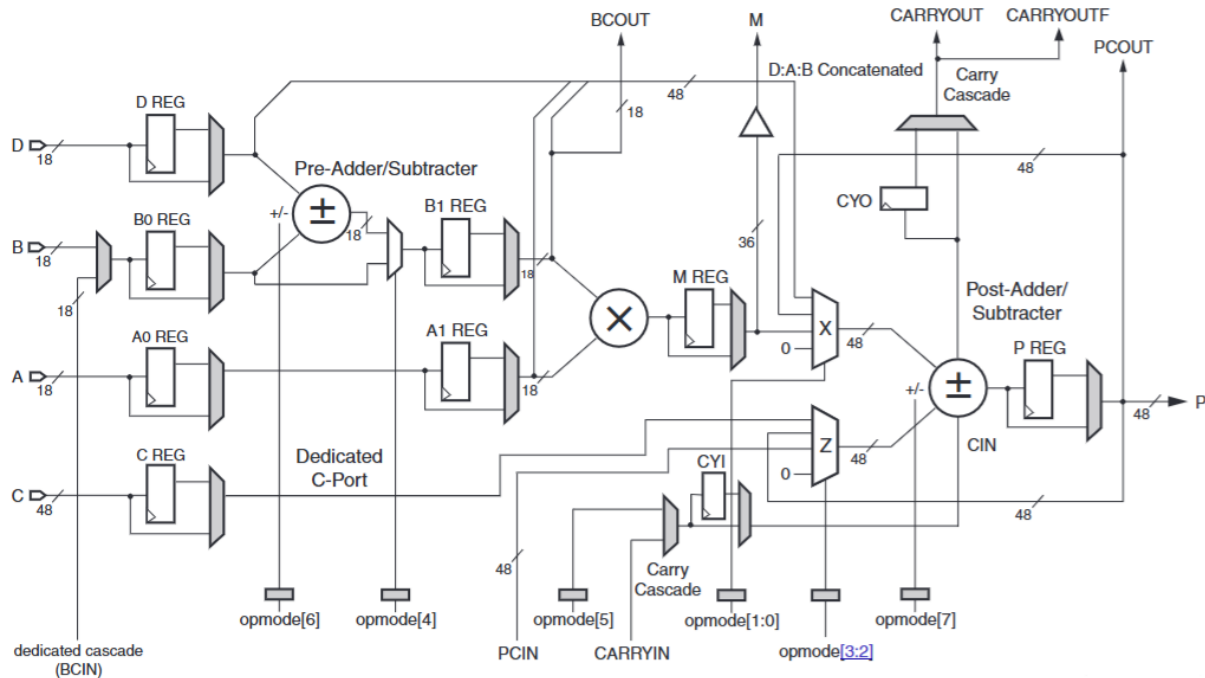
Overview

This project presents the design and implementation of a simplified Digital Signal Processing (DSP) block inspired by the DSP48A1 architecture commonly used in FPGA devices. The objective is to construct a configurable arithmetic unit capable of performing a range of operations—including addition, subtraction, and multiplication—while supporting flexible data routing through multiplexing and pipelining.

To improve performance and reflect realistic hardware design practices, the datapath is divided into multiple pipeline stages. Therefore, the design passed through two phases:

1. Phase 1: a single pipeline register with enable, reset, and two-reset-type capabilities.
2. Phase 2: container parent_dsp block that includes pipeline registers and other constituent blocks.

The design was described in Verilog and verified using directed testbenches to ensure correct functionality across different operating modes. Simulation results were analyzed through waveform inspection to validate the correctness of each pipeline stage and arithmetic operation. The design was further synthesized using Xilinx Vivado to confirm structural integrity and resource mapping within an FPGA environment.



Phase 1: Pipeline register

RTL

```
1  module pipeline_reg #(parameter WIDTH = 48, parameter RSTTYPE = "SYNC")(input clk,
2      input [WIDTH-1:0] in, input rst, input EN,
3      input sel, output [WIDTH-1:0] out);
4      reg [WIDTH-1:0] in_reg;
5  generate
6      if(RSTTYPE == "ASYNC") begin
7          always @(posedge clk or posedge rst) begin
8              if(rst) in_reg <= 0;
9              else if (EN) in_reg <= in;
10         end
11     end else begin
12         always @(posedge clk) begin
13             if(rst) in_reg <= 0;
14             else if (EN) in_reg <= in;
15         end
16     end
17 endgenerate
18 assign out = (sel)? in_reg:in;
19 endmodule
```

Testbench

```
1  module pipeline_tb();
2      reg clk, rst, sel, EN;
3      reg [47:0] in;
4      wire [47:0] out_sync, out_async;
5      //SYNC reset version
6      pipeline_reg #(.WIDTH(48), .RSTTYPE("SYNC")) dut_sync (
7          .clk(clk), .rst(rst), .EN(EN), .sel(sel), .in(in), .out(out_sync)
8      );
9      //ASYNC reset version
10     pipeline_reg #(.WIDTH(48), .RSTTYPE("ASYNC")) dut_async (
11         .clk(clk), .rst(rst), .EN(EN), .sel(sel), .in(in), .out(out_async)
12     );
13
14     initial begin
15         clk = 0;
16         forever #5 clk = ~clk;
17     end
18
```

```

19  initial begin
20      // Initial values
21      rst = 1; EN = 1; sel = 1; in = 10;
22      repeat(2) @(posedge clk);
23      // Release reset
24      rst = 0;
25      // Normal operation
26      repeat(3) @(posedge clk);
27      // Disable register
28      EN = 0; in = 20;
29      repeat(3) @(posedge clk);
30      //Enable it
31      EN = 1;
32      repeat(3) @(posedge clk);
33      //bypass register
34      sel = 0; in = 30;
35      repeat(3) @(posedge clk);
36      // Test ASYNC reset behavior (mid-cycle)
37      sel = 1; @(posedge clk);
38      #2 rst = 1;    // async should react immediately
39      #3 rst = 0;
40
41      repeat(3) @(posedge clk);
42      $stop;
43  end

```

```

45  initial begin
46      $monitor("T=%0t rst=%b EN=%b sel=%b in=%0d | SYNC_out=%0d ASYNC_out=%0d |
47          in_reg_sync=%0d", $time, rst, EN, sel, in, out_sync, out_async, dut_sync.in_reg);
48  end
49
50  endmodule

```

Do file

```

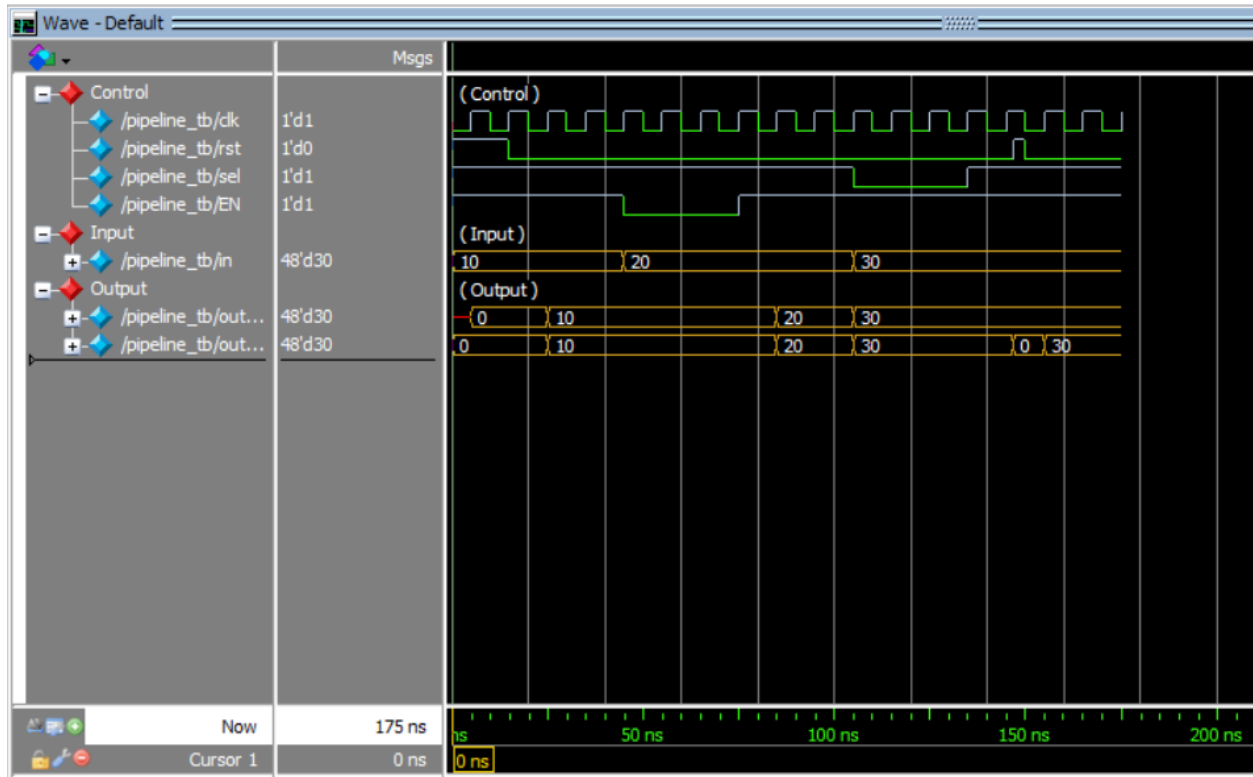
1  # Compile files
2  vlog pipeline_reg.v
3  vlog pipeline_tb.v
4
5  # Start simulation
6  vsim -voptargs=+acc work.pipeline_tb
7
8  # Add waves
9  add wave -position insertpoint sim:/pipeline_tb/*
10
11 # Optional: expand internal DUT signals
12 add wave -position insertpoint sim:/pipeline_tb/dut/*
13
14 # Run simulation
15 run -all

```

Test approach

Test	Procedure	Results
1- Test the rst functionality	- Assert the rst for 2 clock edges, then de-assert it for 3. - EN = 1; in = 10; sel = 1 (registered output).	- out = x because the register's output is not ready yet (waiting for the next clock cycle). - The output is 0 at the second rising clock edge. - The output follows the input = 10 after the third clock cycle.
2- Test enable functionality	Disable the register for 3 clock cycles. Let in = 20;	The register retains its old value (10). After enabling the EN on T=75, the register fetches the new value (20) at the next rising edge (T=85).
3- Selected data source with Sel	Let the data source be 0 (bypass the reg) for 3 clock cycles. Let in = 30.	The output = 30 immediately, while the data registered is still 20 at first (updated on the next clock cycle T=115).
4- Test the reset type behavior	Assert rst mid-cycle. Asynch rst should react immediately, while synch waits for a coming rising edge.	Current time is T=145. After #2, T=147, and asynch_out = 0, while sync_out is still waiting for rising edge. The next rising edge is at T=155, but rst was de-asserted then (after another #3, T=150), so sync_out doesn't capture the early assertion.

Output



```
VSIM 19> run -all
# T=0 rst=1 EN=1 sel=1 in=10 | SYNC_out=x ASYNC_out=0 | in_reg_sync=x
# T=5 rst=1 EN=1 sel=1 in=10 | SYNC_out=0 ASYNC_out=0 | in_reg_sync=0
# T=15 rst=0 EN=1 sel=1 in=10 | SYNC_out=0 ASYNC_out=0 | in_reg_sync=0
# T=25 rst=0 EN=1 sel=1 in=10 | SYNC_out=10 ASYNC_out=10 | in_reg_sync=10
# T=45 rst=0 EN=0 sel=1 in=20 | SYNC_out=10 ASYNC_out=10 | in_reg_sync=10
# T=75 rst=0 EN=1 sel=1 in=20 | SYNC_out=10 ASYNC_out=10 | in_reg_sync=10
# T=85 rst=0 EN=1 sel=1 in=20 | SYNC_out=20 ASYNC_out=20 | in_reg_sync=20
# T=105 rst=0 EN=1 sel=0 in=30 | SYNC_out=30 ASYNC_out=30 | in_reg_sync=20
# T=115 rst=0 EN=1 sel=0 in=30 | SYNC_out=30 ASYNC_out=30 | in_reg_sync=30
# T=135 rst=0 EN=1 sel=1 in=30 | SYNC_out=30 ASYNC_out=30 | in_reg_sync=30
# T=147 rst=1 EN=1 sel=1 in=30 | SYNC_out=30 ASYNC_out=0 | in_reg_sync=30
# T=150 rst=0 EN=1 sel=1 in=30 | SYNC_out=30 ASYNC_out=0 | in_reg_sync=30
# T=155 rst=0 EN=1 sel=1 in=30 | SYNC_out=30 ASYNC_out=30 | in_reg_sync=30
# ** Note: $stop : D:/Digital_Design/New folder/pipeline_tb.v(54)
# Time: 175 ns Iteration: 1 Instance: /pipeline_tb
# Break in Module pipeline_tb at D:/Digital_Design/New folder/pipeline_tb.v line 54
```

Phase 2: Parent DSP

RTL

```
1  module dsp_parent(  
2      input [17:0] A, B, D,  
3      input [47:0] C,  
4      input clk,  
5      input RSTA, RSTB, RSTC, RSTD, RSTM, RSTP, RSTCARRYIN, RSTOPMODE,  
6      input CARRYIN,  
7      input [7:0] OPMODE_in,  
8      input [17:0] BCIN,  
9      input [47:0] PCIN,  
10     input CEA, CEB, CEC, CED, CEM, CEP, CECARRYIN, CEOPMODE,  
11     output [35:0] M,  
12     output [47:0] P,  
13     output [47:0] PCOUT,  
14     output [17:0] BCOUT,  
15     output CARRYOUT  
16 );  
17
```

```
18 parameter A0REG = 0, A1REG = 1, B0REG = 0, B1REG = 1,  
19           CREG = 1, DREG = 1, MREG = 1, PREG = 1,  
20           CARRYINREG = 1, CARRYOUTREG = 1, OPMODEREG = 1;  
21  
22 parameter CARRYINSEL = "OPMODE5"; //& CARRYIN  
23 parameter B_INPUT = "DIRECT"; //& CASCADE  
24 parameter RSTTYPE = "SYNC"; //& ASYNC  
25 wire DREG_sel, B0REG_sel, A0REG_sel, CREG_sel, B1REG_sel, A1REG_sel,  
26      MREG_sel, CARRYINREG_sel, CARRYOUTREG_sel, PREG_sel, OP_MODEREG_sel;  
27 assign {  
28     DREG_sel, B0REG_sel, A0REG_sel, CREG_sel,  
29     B1REG_sel, A1REG_sel, MREG_sel,  
30     CARRYINREG_sel, CARRYOUTREG_sel,  
31     PREG_sel, OP_MODEREG_sel  
32 } = {  
33     DREG, B0REG, A0REG, CREG,  
34     B1REG, A1REG, MREG,  
35     CARRYINREG, CARRYOUTREG,  
36     PREG, OPMODEREG  
37 };
```



```

38 wire [47:0] C0;
39 wire CarryCascade; wire CIN;
40 wire [7:0] OPMODE;
41 wire [17:0] B_in;
42 assign PCOUT = P;
43 wire [17:0] A0, B0, D0, A1, B1;
44 assign BCOUT = B1;
45 wire CARRYOUTF; assign CARRYOUTF = CARRYOUT;
46 wire [47:0] l0;
47 wire COUT; //output from the post-adder/subt
48 assign l0 = {D0[11:0], A1[17:0], B1[17:0]};
49 wire [47:0] x_out;
50 wire [47:0] z_out;
51 //pre adder/subtractor
52 wire [17:0] pre_add_sub_out; assign pre_add_sub_out = (OPMODE[6])? (D0 - B0) : (D0 + B0);
53 wire [17:0] B1_in;
54 //post adder/subtractor
55 wire [47:0] post_add_sub_out;
56 assign post_add_sub_out = (OPMODE[7])? (z_out - (x_out + {{47{1'b0}}, CIN})) : (x_out + z_out);
57 //multiplier
58 wire [35:0] M_in; assign M_in = A1 * B1;

```

```

59 pipeline_reg #(.WIDTH(18), .RSTTYPE(RSTTYPE)) D_REG (.clk(clk), .rst(RSTD), .sel(DREG_sel), .EN(CED), .in(D), .out(D0));
60 pipeline_reg #(.WIDTH(18), .RSTTYPE(RSTTYPE)) B0_REG (.clk(clk), .rst(RSTB), .sel(B0REG_sel), .EN(CEB), .in(B_in), .out(B0));
61 pipeline_reg #(.WIDTH(18), .RSTTYPE(RSTTYPE)) A0_REG (.clk(clk), .rst(RSTA), .sel(A0REG_sel), .EN(CEA), .in(A), .out(A0));
62 pipeline_reg #(.WIDTH(48), .RSTTYPE(RSTTYPE)) C_REG (.clk(clk), .rst(RSTC), .sel(CREG_sel), .EN(CEC), .in(C), .out(C0));
63 pipeline_reg #(.WIDTH(18), .RSTTYPE(RSTTYPE)) B1_REG (.clk(clk), .rst(RSTB), .sel(B1REG_sel), .EN(CEB), .in(B1_in), .out(B1));
64 pipeline_reg #(.WIDTH(18), .RSTTYPE(RSTTYPE)) A1_REG (.clk(clk), .rst(RSTA), .sel(A1REG_sel), .EN(CEA), .in(A0), .out(A1));
65 pipeline_reg #(.WIDTH(36), .RSTTYPE(RSTTYPE)) M_REG (.clk(clk), .rst(RSTM), .sel(MREG_sel), .EN(CEM), .in(M_in), .out(M));
66 pipeline_reg #(.WIDTH(1), .RSTTYPE(RSTTYPE)) CYI (.clk(clk), .rst(RSTCARRYIN), .sel(CARRYINREG_sel), .EN(CECARRYIN), .in(CarryCascade), .out(CIN));
67 pipeline_reg #(.WIDTH(1), .RSTTYPE(RSTTYPE)) CYO (.clk(clk), .rst(RSTCARRYIN), .sel(CARRYOUTREG_sel), .EN(CECARRYIN), .in(COUT), .out(CARRYOUT));
68 pipeline_reg #(.WIDTH(48), .RSTTYPE(RSTTYPE)) P_REG (.clk(clk), .rst(RSTP), .sel(PREG_sel), .EN(CEP), .in(post_add_sub_out), .out(P));
69 pipeline_reg #(.WIDTH(8), .RSTTYPE(RSTTYPE)) OPMODE_REG (.clk(clk), .rst(RSTOPMODE), .sel(OPMODEREG_sel), .EN(CEOPMODE), .in(OPMODE_in), .out(OPMODE));

```

```

70 assign CarryCascade = (CARRYINSEL == "OPMODE5")? OPMODE[5] :
71                      (CARRYINSEL == "CARRYIN")? CARRYIN : 0;
72 assign B_in = (B_INPUT == "DIRECT")? B :
73              (B_INPUT == "CASCADE")? BCIN : 0;
74 //X MUX
75 assign x_out = (OPMODE[1:0] == 2'b11)? l0:
76               (OPMODE[1:0] == 2'b10)? P:
77               (OPMODE[1:0] == 2'b01)? {12{1'b0}, M}: 48'b0;
78 //Z MUX
79 assign z_out = (OPMODE[3:2] == 2'b11)? C0:
80               (OPMODE[3:2] == 2'b10)? P:
81               (OPMODE[3:2] == 2'b01)? PCIN: 48'b0;
82 assign B1_in = (OPMODE[4])? pre_add_sub_out: B0;
83
84 endmodule

```

Testbench

```
1  module dsp_parent_tb();
2      reg [17:0] A, B, D;
3      reg [47:0] C;
4      reg clk;
5      reg RSTA, RSTB, RSTC, RSTD, RSTM, RSTP, RSTCARRYIN, RSTOPMODE;
6      reg CARRYIN;
7      reg [7:0] OPMODE_in;
8      reg [17:0] BCIN;
9      reg [47:0] PCIN;
10     reg CEA, CEB, CEC, CED, CEM, CEP, CECARRYIN, CEOPMODE;
11     wire [35:0] M;
12     wire [47:0] P;
13     wire [47:0] PCOUT;
14     wire [17:0] BCOUT;
15     wire CARRYOUT;
16     wire CARRYOUTF;
```

```
17     dsp_parent dut (.A(A), .B(B), .D(D), .C(C),
18         .clk(clk),
19         .RSTA(RSTA), .RSTB(RSTB), .RSTC(RSTC), .RSTD(RSTD),
20         .RSTM(RSTM), .RSTP(RSTP),
21         .RSTCARRYIN(RSTCARRYIN), .RSTOPMODE(RSTOPMODE),
22         .CARRYIN(CARRYIN),
23         .OPMODE_in(OPMODE_in),
24         .BCIN(BCIN),
25         .PCIN(PCIN),
26         .CEA(CEA), .CEB(CEB), .CEC(CEC), .CED(CED),
27         .CEM(CEM), .CEP(CEP),
28         .CECARRYIN(CECARRYIN), .CEOPMODE(CEOPMODE),
29         .M(M), .P(P), .PCOUT(PCOUT),
30         .BCOUT(BCOUT), .CARRYOUT(CARRYOUT), .CARRYOUTF(CARRYOUTF));
31
32     initial begin
```

```
31
32     initial begin
33         clk = 0;
34         forever #5 clk = ~clk;
35     end
36
```

```

37 initial begin
38 //Path 1 test
39 A = 0; B=0; C=0; D=0; CARRYIN = 0; OPMODE_in = 0; BCIN = 0; PCIN = 0;
40 {RSTA, RSTB, RSTC, RSTD, RSTM, RSTP, RSTCARRYIN, RSTOPMODE} = 0;
41 {CEA, CEB, CEC, CED, CEM, CEP, CECARRYIN, CEOPMODE} = 8'b11111111;
42 @(posedge clk);
43 A = 20; B=10; C=350; D=25; OPMODE_in = 8'b11011101;
44 BCIN = $random;
45 PCIN = $random;
46 CARRYIN = $random;
47 repeat(5) @(negedge clk);
48 if(BCOUT !== 'hf || M !== 'h12c || P !== 'h32 || PCOUT !== 'h32 || CARRYOUT !== 0 || C
ARRYOUTF !== 0) $display("Error detected");
49 else $display("Test 1 succeeded");

```

```

50
51 //Path 2 test
52 A = 20; B = 10; C = 350; D = 25; OPMODE_in = 8'b00010000;
53 BCIN = $random;
54 PCIN = $random;
55 CARRYIN = $random;
56 repeat(3) @(negedge clk);
57 if(BCOUT !== 'h23 || M !== 'h2bc || P !== 0 || PCOUT !== 0 || CARRYOUT !== 0 || CARRYOU
TF !== 0) $display("Error detected");
58 else $display("Test 2 succeeded");
59
60 //Path 3 test
61 A = 20; B = 10; C = 350; D = 25; OPMODE_in = 8'b00001010;
62 BCIN = $random;
63 PCIN = $random;
64 CARRYIN = $random;
65 repeat(3) @(negedge clk);
66 if(BCOUT !== 'ha || M !== 'hc8 || P !== 0 || PCOUT !== 0 || CARRYOUT !== 0 || CARRYOUTF
!== 0) $display("Error detected");
67 else $display("Test 3 succeeded");

```

```

68
69 //path 4 test
70 A = 5; B = 6; C = 350; D = 25; PCIN = 3000; OPMODE_in = 8'b10100111;
71 BCIN = $random;
72 CARRYIN = $random;
73 repeat(3) @(negedge clk);
74 if(BCOUT !== 'h6 || M !== 'h1e || P !== 'hfe6fffec0bb1 || PCOUT !== 'hfe6fffec0bb1 || C
ARRYOUT !== 1 || CARRYOUTF !== 1) $display("Error detected");
75 else $display("Test 4 succeeded");
76
77 $stop;
78
79 end

```

```

80 initial begin
81 $monitor("T=%0t | OPMODE=%b | A=%0d B=%0d D=%0d C=%0d CIN=%b || X=%0d Z=%0d || M=%0d po
    st=%0d P=%0d COUT=%b",
82         $time,
83         OPMODE_in,
84         A, B, D, C, CARRYIN,
85         dut.x_out, dut.z_out,
86         M, dut.post_add_sub_out, P, CARRYOUT);
87 end
88
89 endmodule

```

DO file

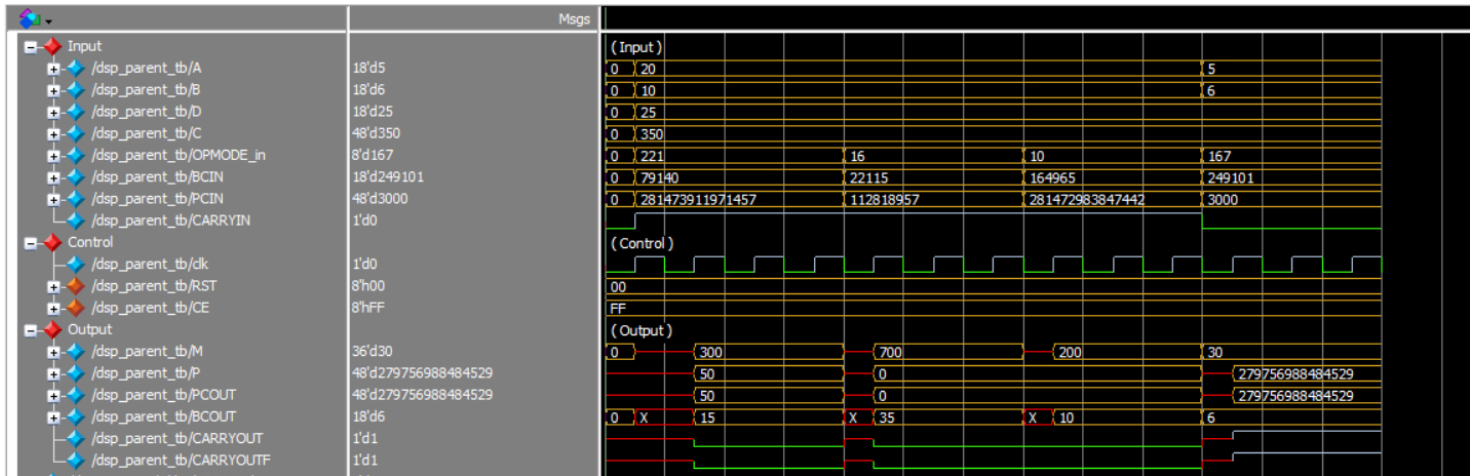
```

1  vdel -all
2  vlib work
3
4  vlog pipeline_reg.v
5  vlog dsp_parent.v
6  vlog pipeline_tb.v
7  vlog dsp_parent_tb.v
8
9  vsim -voptargs=+acc work.dsp_parent_tb
10
11 add wave sim:/dsp_parent_tb/*
12 add wave -r sim:/dsp_parent_tb/dut/*
13
14 run -all

```

Output

QuestaSim



```
# T=0 | OPMODE=00000000 | A=0 B=0 D=0 C=0 CIN=0 || X=x Z=x || M=x post=x P=x COUT=x
# T=5 | OPMODE=11011101 | A=20 B=10 D=25 C=350 CIN=1 || X=0 Z=0 || M=x post=x P=x COUT=x
# T=15 | OPMODE=11011101 | A=20 B=10 D=25 C=350 CIN=1 || X=X Z=350 || M=x post=x P=x COUT=x
# T=25 | OPMODE=11011101 | A=20 B=10 D=25 C=350 CIN=1 || X=200 Z=350 || M=200 post=150 P=x COUT=x
# T=35 | OPMODE=11011101 | A=20 B=10 D=25 C=350 CIN=1 || X=300 Z=350 || M=300 post=50 P=150 COUT=0
# T=45 | OPMODE=11011101 | A=20 B=10 D=25 C=350 CIN=1 || X=300 Z=350 || M=300 post=50 P=50 COUT=0
# Test 1 succeeded
# T=50 | OPMODE=00010000 | A=20 B=10 D=25 C=350 CIN=1 || X=300 Z=350 || M=300 post=50 P=50 COUT=0
# T=55 | OPMODE=00010000 | A=20 B=10 D=25 C=350 CIN=1 || X=0 Z=0 || M=300 post=0 P=50 COUT=0
# T=65 | OPMODE=00010000 | A=20 B=10 D=25 C=350 CIN=1 || X=0 Z=0 || M=300 post=0 P=0 COUT=0
# T=75 | OPMODE=00010000 | A=20 B=10 D=25 C=350 CIN=1 || X=0 Z=0 || M=700 post=0 P=0 COUT=0
# Test 2 succeeded
# T=80 | OPMODE=00001010 | A=20 B=10 D=25 C=350 CIN=1 || X=0 Z=0 || M=700 post=0 P=0 COUT=0
# T=105 | OPMODE=00001010 | A=20 B=10 D=25 C=350 CIN=1 || X=0 Z=0 || M=200 post=0 P=0 COUT=0
# Test 3 succeeded
# T=110 | OPMODE=10100111 | A=5 B=6 D=25 C=350 CIN=0 || X=0 Z=0 || M=200 post=0 P=0 COUT=0
# T=115 | OPMODE=10100111 | A=5 B=6 D=25 C=350 CIN=0 || X=1717988229126 Z=3000 || M=200 post=279756988484530 P=0 COUT=0
# T=125 | OPMODE=10100111 | A=5 B=6 D=25 C=350 CIN=0 || X=1717988229126 Z=3000 || M=30 post=279756988484529 P=279756988484530 COUT=1
# T=135 | OPMODE=10100111 | A=5 B=6 D=25 C=350 CIN=0 || X=1717988229126 Z=3000 || M=30 post=279756988484529 P=279756988484529 COUT=1
# Test 4 succeeded
# ** Note: $stop : dsp_parent_tb.v(77)
# Time: 140 ns Iteration: 1 Instance: /dsp_parent_tb
# Break in Module dsp_parent_tb at dsp_parent_tb.v line 77
```

QuestaLint

Warning messages regarding the constant values of the Sel selector signal for each register were observed. This results from the use of constant parameters as default values at the beginning of the RTL in an attempt to control the datapaths.

```
Check      : mux_select_const
Module     : pipeline_reg
Signal(s)  : M_REG.out
Reference  : D:/Rady/pipeline_reg.v,18
Message    : Constant value drives mux select pin. Signal M_REG.out, Module pipeline_reg, File D:/
             Rady/pipeline_reg.v, Line 18.
Status     : uninspected
Owner      : unassigned
```

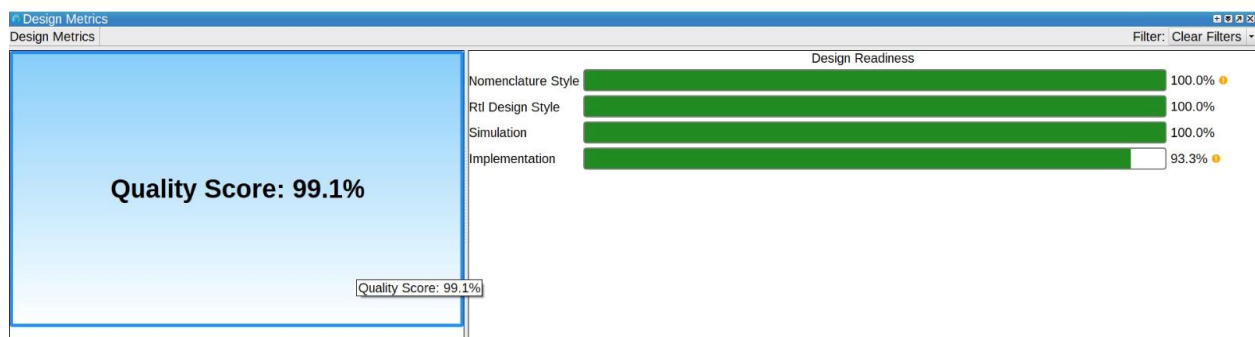
```
vlog D:/Rady/dsp_parent.v D:/Rady/pipeline_reg.v -work work
-- Compiling module dsp_parent
-- Compiling module pipeline_reg
```

Top level modules:

dsp_parent

End time: 15:07:23 on Mar 23,2026, Elapsed time: 0:00:00

Errors: 0, Warnings: 0

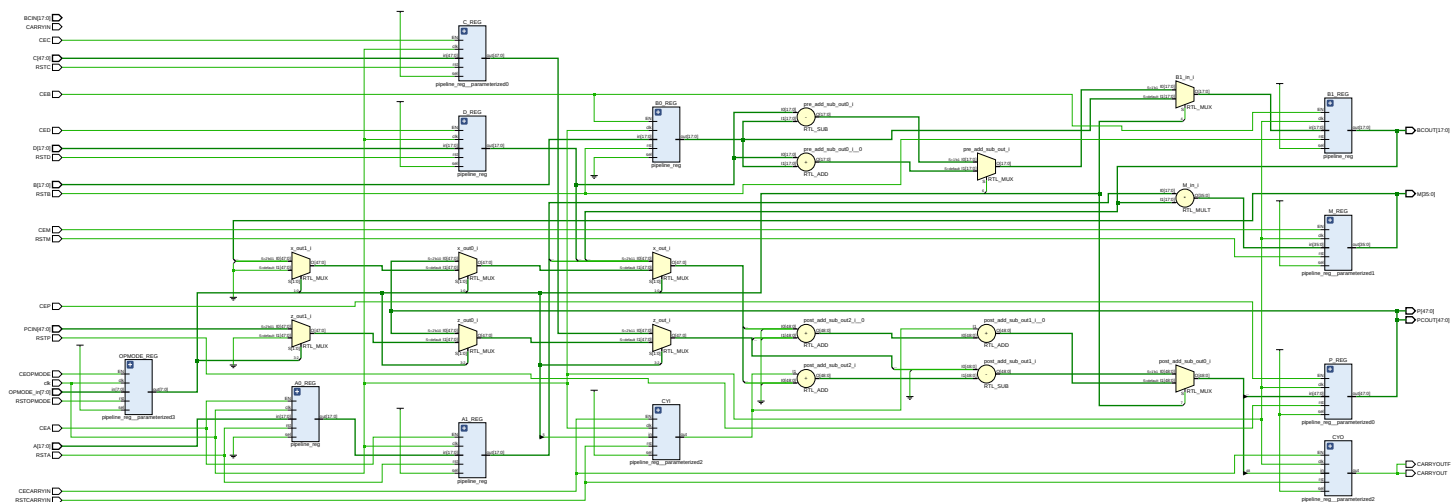


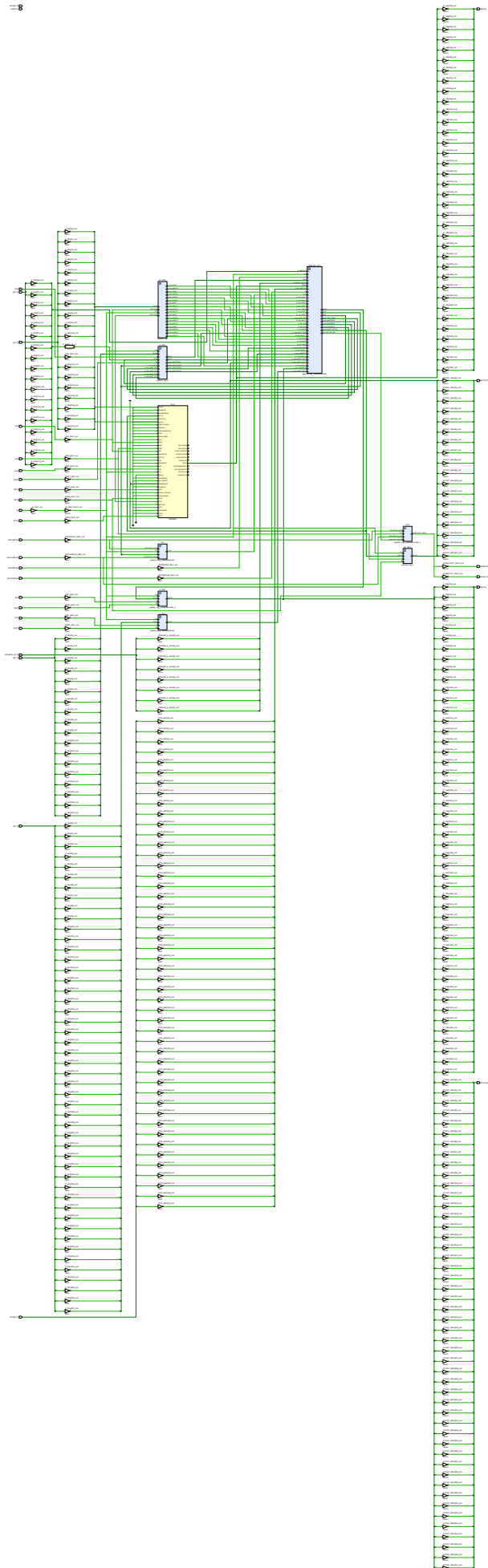
Vivado

A timing constraint defining a 100 MHz clock on pin W5 was specified using a dedicated constraints file. To accommodate the high I/O requirements of the design, the xc7a200tffg1156-3 device was selected. Additionally, schematic views from both elaboration and synthesis stages were documented to illustrate the implemented DSP architecture within the FPGA fabric.

Constraints file (brief)

```
1  ## This file is a general .xdc for the Basys3 rev B board
2  ## To use it in a project:
3  ## - uncomment the lines corresponding to used pins
4  ## - rename the used ports (in each line, after get_ports) according to the top level
   signal names in the project
5
6  ## Clock signal
7  set_property -dict { PACKAGE_PIN W5    IOSTANDARD LVCMOS33 } [get_ports clk]
8  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
9
```





Utilization Report

Tcl ConsoleMessagesLogReportsDesign RunsUtilization x

Q

Hierarchy

Hierarchy

Summary

▼ Slice Logic

▼ Slice LUTs (<1%)

LUT as Logic (<1%)

▼ Slice Registers (<1%)

Register as Flip Flop (<1)

Memory

▼ DSP

▼ DSPs (<1%)

DSP48E1 only

▼ IO and GT Specific

▼ Bonded IOB (65%)

IOB Master Pads

▼ Clocking

BUFGCTRL (3%)

Specific Feature

Primitives

Black Boxes

Instantiated Netlists

1

Name

▼

▼ N dsp_parent

A1_REG (pipeline_reg_

B1_REG (pipeline_reg_

C_REG (pipeline_reg_

CYI (pipeline_reg_pa

CYO (pipeline_reg_p

D_REG (pipeline_reg_

OPMODE_REG (pipeli

P_REG (pipeline_reg_

Slice LUTs
(134600)

231

0

0

0

1

0

0

0

0

229

0

Slice Registers
(269200)

160

18

18

48

1

1

0

0

18

8

48

DSPs (740)

1

0

0

0

0

0

0

0

0

0

0

Bonded IOB
(500)

327

0

0

0

0

0

0

0

0

0

0

BUFGCTRL
(32)

1

0

0

0

0

0

0

0

0

0

0

utilization_1

Timing Summary

Timing warnings related to missing I/O delay constraints were observed. These do not indicate functional errors; rather, they reflect the absence of external timing specifications, which are outside the scope of this design.

The screenshot shows the 'Design Timing Summary' report. The left sidebar contains a tree view with the following items: General Information, Timer Settings, Design Timing Summary (selected), Clock Summary (1), Check Timing (326), Intra-Clock Paths, Inter-Clock Paths, Other Path Groups, User Ignored Paths, and Unconstrained Paths. The main content area is titled 'Design Timing Summary' and contains three columns: Setup, Hold, and Pulse Width. Each column lists various timing metrics and their values.

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 4.813 ns	Worst Hold Slack (WHS): 0.261 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 106	Total Number of Endpoints: 106	Total Number of Endpoints: 181

All user specified timing constraints are met.

The screenshot shows the 'Check Timing' report. The left sidebar contains a tree view with the following items: General Information, Timer Settings, Design Timing Summary, Clock Summary (1), Check Timing (326) (selected), no_clock (0), constant_clock (0), pulse_width_clock (0), unconstrained_internal_endpoints, no_input_delay (174), no_output_delay (152), multiple_clock (0), generated_clocks (0), loops (0), partial_input_delay (0), partial_output_delay (0), latch_loops (0), Intra-Clock Paths, Inter-Clock Paths, Other Path Groups, User Ignored Paths, and Unconstrained Paths. The main content area is titled 'Check Timing' and contains a table with three columns: Timing Check, Count, and Worst Severity.

Timing Check	Count	Worst Severity
no_input_delay	174	High
no_output_delay	152	High
no_clock	0	
constant_clock	0	
pulse_width_clock	0	
unconstrained_internal_endpoints	0	
multiple_clock	0	
generated_clocks	0	
loops	0	
partial_input_delay	0	
partial_output_delay	0	
latch_loops	0	

Messages

Reported warnings are related to unused signals (due to muxed datapath), missing constraints (I/O), and device configuration settings (e.g., voltage mismatch), and do not impact the functional correctness of the design.

